

GovPay Desk Technical Documentation

Architecture, data model, routes and conventions for the codebase.

System	Audience	Revised
GovPay Desk	Developers and maintainers	8 August 2026

CONTENTS

1. Stack
2. Getting started
3. Project layout
4. Data model
5. Authorization
6. Request lifecycle: draft → paid
7. Services
8. AI features
9. Routes
10. Frontend
11. Querying conventions
12. Testing
13. Known gaps and technical debt

1. Stack

LAYER	TECHNOLOGY
Backend	Laravel 12, PHP 8.2+
Frontend	React 19 + TypeScript, via Inertia 2
Styling	Tailwind CSS 4, shadcn/ui (Radix primitives)
Build	Vite 7
Auth	Laravel Fortify (incl. two-factor)
Routing helpers	Laravel Wayfinder — generates typed route helpers into <code>resources/js/routes/</code> and <code>resources/js/actions/</code>
Charts	Recharts
Database	SQLite by default (<code>DB_CONNECTION=sqlite</code>)
AI	<code>anthropic-ai/sdk</code> — Claude API, optional
Tests	Pest 4
Formatting	Pint (PHP), Prettier + ESLint (TS)

There is no separate API layer — every page is an Inertia response rendered by a controller. The two AI endpoints are the only JSON responses.

2. Getting started

```
composer install
npm install
cp .env.example .env
php artisan key:generate
php artisan migrate:fresh --seed
composer dev          # server + queue + logs + vite, concurrently
```

Then open `http://127.0.0.1:8000`.

Seeded accounts

All seeded accounts use the password `password`.

EMAIL	ROLE	PURPOSE
<code>ota@example.com</code>	Administrator	Full access; can approve own vouchers
<code>ama@example.com</code>	Finance Officer	Prepares, submits, pays
<code>kofi@example.com</code>	Finance Officer	Same
<i>(2 generated users in Administration)</i>	Approver	Approve/reject others' vouchers
<i>(4 generated users in Procurement)</i>	Viewer	Read-only

The approver and viewer accounts are generated by factory, so their emails vary per seed — look them up with `php artisan tinker` or the Users page. To walk the core loop, sign in as `ama@example.com` to prepare and submit, then as `ota@example.com` to approve.

Optional: AI features

```
ANTHROPIC_API_KEY=sk-ant-...
ANTHROPIC_MODEL=c laude-opus-5      # default
ANTHROPIC_TIMEOUT=45                 # seconds, default
```

Without a key every AI feature reports itself unavailable and the rest of the application is unaffected. See section 8.

3. Project layout

app/	
Http/Controllers/	one per feature area; all render Inertia
Http/Requests/	validation, grouped by model
Models/	Account, AppNotification, Department, Document, LedgerEntry, Memo, PaymentVoucher, SystemLog, User
Policies/	PaymentVoucherPolicy – the whole authorization story
Services/	LedgerPoster, AuditLogger, Notifier, VoucherIntelligence
config/anthropic.php	AI configuration
database/	
migrations/	see section 4
seeders/	DatabaseSeeder builds a full demo dataset
resources/js/	
pages/	one folder per feature, matching route names
components/	app shell, sidebar, shared widgets
components/ui/	shadcn primitives
layouts/	app, auth and settings layouts
routes/, actions/	Wayfinder-generated – do not hand-edit
tests/Feature/	LedgerPosting, VoucherPolicy, VoucherWorkflow, PagesLoad, Dashboard, Auth/*, Settings/*

All models use **ULID primary keys** (`HasUlid`), not auto-increment integers.

`PaymentVoucher`, `Memo` and `User` are soft-deletable.

4. Data model

Migration history

MIGRATION	CONTENTS
0001_01_01_*	users, cache, jobs (framework defaults)
2025_08_14_170933	two-factor columns on users
2026_01_27_092633	staff/approval fields on users
2026_01_27_093232	departments
2026_01_27_153134	payment_vouchers, voucher_documents, voucher_approvals
2026_01_28_164205	memos
2026_08_03_095100	drops unused MVP tables and columns
2026_08_08_100000	role on users; accounts, ledger_entries, documents, system_logs, app_notifications

Read 2026_08_03_095100 before trusting the January migrations. It drops voucher_documents and voucher_approvals entirely (their models VoucherDocument and VoucherApproval are also deleted), along with users.last_login_at, last_login_ip, approval_level, approval_limit, password_changed_at, payment_vouchers.internal_notes and memos.printed_at.

Document storage is now the polymorphic documents table; approval state lives on the voucher row itself.

One inconsistency to be aware of: PaymentVoucher::\$fillable still lists internal_notes, whose column was dropped. It is never written to, so nothing breaks, but it is a dead entry.

Core tables

payment_vouchers — the central record.

GROUP	COLUMNS
Identity	<code>voucher_number</code> (unique), <code>voucher_date</code>
Payee	<code>payee_name</code> , <code>payee_account_number</code> , <code>payee_bank</code> , <code>payee_phone</code>
Payment	<code>description</code> , <code>amount</code> (decimal 15,2), <code>payment_method</code> , <code>cheque_number</code> , <code>payment_reference</code>
Budget	<code>budget_line</code> , <code>budget_code</code> , <code>department_id</code>
State	<code>status</code> — <code>draft</code> \ <code>pending</code> \ <code>approved</code> \ <code>rejected</code> \ <code>paid</code>
Actors	<code>created_by</code> , <code>approved_by</code> , <code>rejected_by</code> , <code>paid_by</code>
Timing	<code>submitted_at</code> , <code>approved_at</code> , <code>rejected_at</code> , <code>paid_at</code>
Notes	<code>rejection_reason</code>

`department_id` and `created_by` use `restrictOnDelete` — a department or user with vouchers cannot be deleted, preserving the historical record.

`memos` — `memo_number` (unique), `memo_date`, `subject`, `body`, `to_name`, `to_designation`, `from_name`, `from_designation`, `department_id`, nullable `voucher_id`, `created_by`, `status` (`draft` | `finalized` | `printed`). There is no `printed_at` — it was created then dropped, so "printed" is only a status, with no timestamp recording when.

`accounts` — `code` (unique), `name`, `type` (asset/liability/equity/revenue/expense), `is_active`.

`ledger_entries` — one row per side of a double entry: `reference`, `entry_date`, `account_id`, `description`, `debit`, `credit`, nullable `payment_voucher_id` (the origin document), `created_by`.

`documents` — polymorphic (`documentable_type` / `documentable_id`): `name`, `path`, `mime_type`, `size`, `uploaded_by`.

`system_logs` — `action`, `description`, polymorphic subject, `user_id`, `ip_address`. Append-only by convention; nothing in the app updates or deletes.

`app_notifications` — `user_id`, `type`, `title`, `body`, `link`, `read_at`.

Reference-number generation

Both voucher and memo numbers are generated per year by reading the last record and incrementing its trailing three digits:

```
'PV-' . date('Y') . '-' . str_pad($sequence, 3, '0', STR_PAD_LEFT)
'MEMO-' . date('Y') . '-' . str_pad($sequence, 3, '0', STR_PAD_LEFT)
```

This is read-then-write with no lock or unique retry, so two simultaneous creations can collide on the unique index. Acceptable at this scale; replace with a sequence table or a retry on constraint violation before real concurrent use.

5. Authorization

All authorization for vouchers lives in `PaymentVoucherPolicy.php`. Roles are constants on `User` (`admin`, `approver`, `finance_officer`, `viewer`), fixed in code — there is no roles table.

ABILITY	REQUIRES STATUS	REQUIRES ROLE	EXTRA
<code>viewAny</code>	—	any	user must be active
<code>create</code>	—	admin, finance officer	—
<code>update</code>	draft or rejected	admin, finance officer	—
<code>delete</code>	draft	admin, finance officer	—
<code>submit</code>	draft	admin, finance officer	—
<code>review</code>	pending	admin, approver	preparer cannot review own unless admin
<code>markPaid</code>	approved	admin, finance officer	—

Two invariants are worth stating explicitly because the whole design rests on them:

1. **Immutability after approval.** `update` returns false for any status other than `draft` or `rejected`, so approved and paid vouchers cannot be edited. `VoucherPolicyTest` asserts this against the route, not just the policy.
2. **Separation of duties.** `review` returns false when `$voucher->created_by === $user->id` unless the user is an admin.

Memos, users, departments and ledger pages have **no policy**. They are gated only by the `auth` + `verified` middleware, so any authenticated user can reach them. If role-gating those areas matters, that is new work.

6. Request lifecycle: draft → paid

```
store()      → status draft, created_by = actor, number generated
               → AuditLogger 'voucher.created'

submit()     → authorize('submit') → status pending, submitted_at = now
               → AuditLogger 'voucher.submitted'
               → Notifier::toApprovers(...)

review()     → authorize('review') → approve: status approved, approved_by/_at
                                   reject:  status rejected, rejected_by/_at,
                                   rejection_reason (required)
               → AuditLogger 'voucher.approved' | 'voucher.rejected'
               → Notifier::toUser(creator, ...)
```



```
markPaid()   → authorize('markPaid') → status paid, paid_by/_at
               → LedgerPoster::post($voucher->refresh())
               → AuditLogger 'voucher.paid'
               → Notifier::toUser(creator, ...)
```

`markPaid` calls `refresh()` before posting because `LedgerPoster` reads `status` and `paid_at` off the model and would otherwise see stale values.

7. Services

LedgerPoster

Posts a paid voucher as a balanced double entry inside a transaction:

```
Dr  expense account    (mapped from budget_line)
Cr  1100 Cash | 1200 Bank  (from payment_method)
```

Three guards, in order: it returns early unless `status === 'paid'`; it returns early if a `LedgerEntry` already exists for the voucher (**idempotency** — this is why re-seeding does not duplicate); and it returns early if either account lookup fails.

Budget lines are free text, so `BUDGET_LINE_ACCOUNTS` maps substrings to codes with `str_contains`, falling back to `5900 General Expenses`:

SUBSTRING	CODE
office supplies, stationery	5100
salaries, payroll	5200
travel, transport	5300
utilities	5400
maintenance	5500
(no match)	5900

Settlement: `bank_transfer` → `1200`, everything else → `1100`.

To extend: add to `BUDGET_LINE_ACCOUNTS` and ensure the account exists in the seeder. The structurally correct fix is to replace the free-text `budget_line` with a foreign key to `accounts`; the string mapping is a deliberate MVP shortcut.

AuditLogger

`AuditLogger::record($action, $description, ?$subject)` writes a `SystemLog` row with the authenticated user and request IP. Static, no interface, called directly from controllers.

Notifier

`Notifier::toUser(...)` creates one `AppNotification`.
`Notifier::toApprovers(...)` fans out to every **active** user with role admin or approver. In-app only — no mail, no queue.

VoucherIntelligence

See section 8.

8. AI features

VoucherIntelligence.php wraps the Claude API for two advisory features. The governing design rule is that **AI is an assist, never a dependency** — every path degrades to null and the caller falls back to the manual flow.

`isEnabled()` — true only when `config('anthropic.api_key')` is filled. Controllers pass this to the frontend as `aiEnabled` so the UI can hide or explain the feature.

`reviewVoucher(PaymentVoucher): ?array` — loads the last 20 paid vouchers from the same department as context and asks Claude to flag anything an approver should check. Returns `{risk: low|medium|high, summary: string, findings: string[]}`.

`draftMemo(PaymentVoucher): ?array` — drafts a memo subject and body from a paid voucher. Returns `{subject: string, body: string}`.

Both use structured output (`outputConfig` with a JSON schema), so the response shape is enforced by the API rather than parsed hopefully. Both prompts explicitly instruct the model to use only the supplied facts and not to invent references, dates or signatories.

Failure handling: every call is wrapped in `try/catch(Throwable)`. A thrown exception, a `refusal` stop reason, or a non-array decode all log a warning and return null. The controller then returns `{available: false, message: ...}`, distinguishing "not configured on this deployment" from "temporarily unavailable".

Endpoints:

ROUTE	AUTH	NOTES
POST /payment-vouchers/{voucher}/ai-review	<code>authorize('review')</code>	Logs <code>voucher.ai_reviewed</code> on success
POST /memos/ai-draft/{voucher}	voucher must be <code>paid</code>	Logs <code>memo.ai_drafted</code> on success

Known gap: `ai-draft` has no frontend caller. Wayfinder has generated route helpers for it, but no component on `memos/create.tsx` invokes them, so memo drafting is unreachable from the UI. `ai-review` is wired, on `payment-vouchers/pending.tsx`.

Note that `ai-draft` is authorized only by voucher status, not by policy or role — any authenticated user can spend an API call against any paid voucher. Worth tightening if the key is production-billed.

9. Routes

Defined in routes/web.php, all behind `auth` + `verified` except the root redirect.
Settings routes are in routes/settings.php.

METHOD	URI	NAME
GET	/	home → redirects to login
GET	dashboard	dashboard
—	departments (resource)	departments.*
GET	payment-vouchers	payment-vouchers.index
GET	payment-vouchers/create	payment-vouchers.create
GET	payment-vouchers/pending	payment-vouchers.pending
GET	payment-vouchers/rejected	payment-vouchers.rejected
GET	payment-vouchers/{voucher}/edit	payment-vouchers.edit
POST	payment-vouchers	payment-vouchers.store
PUT	payment-vouchers/{voucher}	payment-vouchers.update
DELETE	payment-vouchers/{voucher}	payment-vouchers.destroy
POST	payment-vouchers/{voucher}/submit	payment-vouchers.submit
POST	payment-vouchers/{voucher}/review	payment-vouchers.review
POST	payment-vouchers/{voucher}/mark-paid	payment-vouchers.mark-paid
POST	payment-vouchers/{voucher}/ai-review	payment-vouchers.ai-review
GET	memos	memos.index
GET	memos/create	memos.create
POST	memos	memos.store
POST	memos/{memo}/finalize	memos.finalize
POST	memos/{memo}/print	memos.print
POST	memos/ai-draft/{voucher}	memos.ai-draft
GET	ledgers	ledgers.index
GET	ledgers/transactions	ledgers.transactions
GET	ledgers/chart-of-accounts	ledgers.chart-of-accounts

METHOD	URI	NAME
GET	<code>financial-reports/monthly</code>	<code>financial-reports.monthly</code>
GET	<code>financial-reports/department</code>	<code>financial-reports.department</code>
GET	<code>users</code> , <code>users/create</code> , <code>users/{id}/edit</code>	<code>users.*</code>
POST/PUT	<code>users</code> , <code>users/{id}</code>	<code>users.store</code> , <code>users.update</code>
GET	<code>documents</code>	<code>documents.index</code>
GET	<code>system-logs</code>	<code>system-logs.index</code>
GET	<code>roles-permissions</code>	<code>roles-permissions.index</code>
GET	<code>notifications</code>	<code>notifications.index</code>
POST	<code>notifications/{notification}/read</code> , <code>notifications/read-all</code>	<code>notifications.read</code> , <code>notifications.read-all</code>

There is deliberately **no** `payment-vouchers.show` — lists link to Edit.

`payment-vouchers/create`, `/pending` and `/rejected` are declared **before** `payment-vouchers/{voucher}/edit`, so ordering matters if you add routes there.

Wayfinder

After changing routes, regenerate the typed helpers:

```
php artisan wayfinder:generate --with-form
```

The `--with-form` **flag is required**. Without it the `.form` variants that the Fortify auth pages depend on disappear and the build breaks.

10. Frontend

Pages live in `resources/js/pages/` mirroring route names —

`payment-vouchers/pending.tsx` serves `payment-vouchers.pending`. Controllers name them explicitly in `Inertia::render()`.

Navigation is defined statically in `app-sidebar.tsx`, in three groups: Financial Operations, Reports & Documents, Administration. **The sidebar does not consult roles** — every authenticated user sees every link, and authorization is enforced server-side when the action is attempted.

UI conventions

Match these when adding pages:

- **Say a thing once.** Page title, card title and section heading should not repeat each other. A status badge on a page already filtered to that status adds nothing.
- **Card padding is set once**, on `<Card className="py-5">`. Do not add `pt-6` to `CardContent` — shadcn's `Card` already has `py-6` and the two stack into a dead band.
- **Form spacing scale:** `space-y-5` between sections, `space-y-2` heading to fields, `gap-y-3.5` between field rows, `space-y-1.5` label to input.
- **Money** is right-aligned, `tabular-nums`, two decimals, em-dash for zero. Tables showing money show totals.
- **Every list** needs an empty state that distinguishes "nothing here yet" from "nothing matches your filter", and offers a way out of the filter.
- **Every paginated list renders pagination.** A count reading "10 of 13" with no way to reach the other 3 is a bug.
- **Errors** appear inline *and* in a summary at the top of long forms, each summary line linking to its field.
- **Dates** use `en-GB` formatting (`3 August 2026`).
- **Colour is never the only signal** — pair it with an icon or text.

11. Querying conventions

Index controllers follow a consistent shape: `when($request->filled(...))` filters, `latest()`, `paginate()`, `withQueryString()`, and a `filters` prop echoing `$request->only([...])` so the UI can render current filter state.

Two things to be aware of:

- **Search is LIKE-based and unindexed.** Fine at this data size; it will not scale.
- `LedgerController::transactions` and `FinancialReportController` use **SQLite functions directly** — `strftime('%m', paid_at)`, `strftime('%Y', paid_at)`. These will break on MySQL or Postgres. Porting the

database means rewriting those aggregates.

- `LedgerController::transactions` computes totals over the **whole filtered set** with a second query, not just the visible page — duplicating the filter chain. If you change one filter, change both.

12. Testing

```
php artisan test          # 81 tests, 199 assertions
npx tsc --noEmit          # typecheck
npx vite build            # production build
./vendor/bin/pint         # PHP formatting
npm run lint              # ESLint
composer test             # config:clear + pint --test + artisan test
```

Coverage of the domain logic:

TEST FILE	COVERS
<code>LedgerPostingTest</code>	balanced entries, budget-line → account mapping, fallback account, idempotency
<code>VoucherPolicyTest</code>	every policy ability, separation of duties, and route-level tampering against a paid voucher
<code>VoucherWorkflowTest</code>	end-to-end draft → paid with ledger posting; rejection reason + notification; cheque-number requirement
<code>PagesLoadTest</code>	smoke test that every page returns 200
<code>DashboardTest</code> , <code>Auth/*</code> , <code>Settings/*</code>	starter-kit auth and settings behaviour

`VoucherIntelligence` has **no test coverage** — the AI paths, including the degradation-to-null behaviour, are untested. The constructor accepts an injected `?Client`, so it is mockable if you add them.

13. Known gaps and technical debt

Ordered roughly by how likely each is to bite:

1. **Documents cannot be uploaded.** The `documents` table, `Document` model and index page exist; there is no store endpoint, no file handling, no validation. Records only appear via the seeder.
2. **AI memo drafting is unreachable** from the UI (section 8).
3. `ai-draft` **is not role-authorized** — status check only.
4. **No policies outside vouchers.** Memos, users, departments and ledgers are gated only by `auth`.
5. **Reference-number generation can race** (section 4).
6. **SQLite-specific date SQL** in reports and ledger queries (section 11).
7. **Ledger is one-directional** — no manual journals, no reversals, no period close. A mistaken payment cannot be corrected in the ledger.
8. `budget_line` **is free text**, mapped by substring. Should be a foreign key.
9. **Search is unindexed** `LIKE`.
10. **Admin-created users get** `password` with no invite or reset flow.
11. **Memos cannot be edited or deleted**, and produce no printable output.
12. **No** `payment-vouchers.show` **route**.
13. **GIFMIS is not implemented** and has been removed from the codebase.
14. **Dead** `internal_notes` **entry** in `PaymentVoucher::$fillable` for a column that no longer exists (section 4).